

Reviewer Pack — Secant Rank of Disjointness Communication Matrix Bounds Rand...

Entry c22dcadfaac9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 13:48:36 UTC

Secant Rank of Disjointness Communication Matrix Bounds Randomized Complexity

Entry ID: c22dcadfaac9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 13:48:36 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Secant Varieties of Veronese Reembeddings)

Field B (complexity object): Communication Complexity of Disjointness

Statement:

For any $n \times n$ disjointness matrix M , the secant rank $\text{sr}(M)$ satisfies $\text{sr}(M) \geq \Omega(n)$. For random instances, $\text{sr}(M) = \Theta(n)$ with probability ≥ 0.95 over 30 seeds.

Rationale (proposer's reasoning):

Secant rank captures the minimal number of rank-1 tensors needed to approximate M . High secant rank implies structural complexity, aligning with $\Omega(n)$ lower bounds for disjointness. Veronese reembeddings provide a natural framework to quantify this geometric constraint.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 068f90180959644f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_disjointness_matrix(n):
    M = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            M[i][j] = random.choice([0, 1])
            M[j][i] = M[i][j]
    return M

def matrix_multiplication(A, B):
    m, k, n = len(A), len(B[0]), len(B)
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for l in range(k):
                C[i][j] += A[i][l] * B[l][j]
    return C

def svd(A):
    m, n = len(A), len(A[0])
    U = [[0] * n for _ in range(m)]
    S = [0] * min(m, n)
    V = [[0] * m for _ in range(n)]

    # Compute A^T * A
    AT_A = matrix_multiplication(transpose(A), A)

    # Compute eigenvalues and eigenvectors of A^T * A
    eigenvals, eigenvecs = power_iteration(AT_A, n)

    # Sort eigenvalues and corresponding eigenvectors
    for i in range(n):
        S[i] = eigenvals[i]
        U[i] = [eigenvecs[j][i] for j in range(m)]

    # Compute V using A * U
    V = matrix_multiplication(A, U)
```

```

    return U, S, V

def transpose(M):
    m, n = len(M), len(M[0])
    T = [[0] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            T[j][i] = M[i][j]
    return T

def power_iteration(A, k):
    eigenvals = [1.0] * k
    eigenvcs = [[random.random() for _ in range(len(A))] for _ in range(k)]

    for _ in range(100): # Number of iterations
        for i in range(k):
            v = matrix_multiplication(A, eigenvcs[i])
            norm = sum(x * x for x in v) ** 0.5
            eigenvals[i] = sum(v[j] * eigenvcs[j][i] for j in range(len(v)))
            eigenvcs[i] = [x / norm for x in v]

    return eigenvals, eigenvcs

def secant_rank(M):
    n = len(M)
    M_tensor = []
    for i in range(n):
        for j in range(n):
            M_tensor.append([M[i][k] * M[j][k] for k in range(n)])

    U, S, V = svd(M_tensor)
    rank = sum(1 for s in S if abs(s) > 1e-6)
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    M = generate_disjointness_matrix(n)

    sr_M = secant_rank(M)
    if sr_M < 0.8 * n:
        return {
            "metric_name": "secant rank",
            "metric_value": sr_M,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"n={n}, sr(M)={sr_M}"
        }

    return {
        "metric_name": "secant rank",
        "metric_value": sr_M,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='n={results[0]['instances_tested']}', sr(M)={results[0]['sr_M']}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eeb7df7f.py", line 118, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eeb7df7f.py", line 94, in run_trial
    sr_M = secant_rank(M)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eeb7df7f.py", line 85, in secant_rank
    U, S, V = svd(M_tensor)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eeb7df7f.py", line 42, in svd
    AT_A = matrix_multiplication(transpose(A), A)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eeb7df7f.py", line 32, in matrix_multiplication
    C[i][j] += A[i][l] * B[l][j]
              ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before collecting data; cannot assess support fraction or counterexamples. | next: Debug the matrix multiplication implementation to fix index out of range error

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109665
2	propose	ollama_remote	qwen3:8b	0	0	33551
3	preregistration	ollama_remote	qwen3:8b	0	0	25099
4	novelty	ollama_remote	qwen3:8b	0	0	20753
5	novelty	ollama_remote	qwen3:8b	0	0	14678
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15162
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12909
8	judge	ollama_remote	qwen3:8b	0	0	16641

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 248458 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/c22dcadfaac9.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c22dcadfaac9.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c22dcadfaac9.tar.gz (if generated)